

Analysis: Fair Warning

This turned out to be the hardest problem in the qualification round. One of the reasons may have been that the solution involves big arithmetic precision. Using big numbers in timed programming contests is sometimes not considered fair because some languages like python or java have internal libraries that deal with this while for other languages you may have needed to write your own or search for an external one like the GNU Multi-Precision Library. Considering that the qualification round was 24 hours long we took this chance to give you a warning and one which is fair ... that big numbers are fair game for now on, so have a library on hand!

This problem talks about divisors and multiples so it hints at using the greatest common divisor concept in some way. To solve the problem we need to find T and afterwards we can easily find y .

Let's simplify the problem and just look at two numbers a and b . In this case we need to find the largest T so that some positive y exists where $a + y$ and $b + y$ are multiples of T . So $(a + y) \bmod T = (b + y) \bmod T$ which means that $(a - b) \bmod T = 0$. Thus T must be a divisor of $|a - b|$.

Coming back to the problem with N numbers, we have proved that T must divide every $|t_i - t_j|$. This means that the solution is the greatest common divisor of all $|t_i - t_j|$ numbers. A simple observation can improve our algorithm from $O(N^2)$ to $O(N)$ iterations of the Euclidean algorithm. Let's say $a \leq b \leq c$, then $\gcd(b - a, c - a) = \gcd(b - a, c - a, c - b)$. To prove this we look at the first iteration of the Euclidean algorithm. Since $a \leq b \leq c$ it means that $b - a \leq c - a$ so in the first step we subtract $b - a$ from $c - a$: $\gcd(b - a, c - a) = \gcd(c - b, b - a)$, this proves our observation. Now we can just compute the greatest common divisor of all $|t_i - t_1|$ to find T .

Here's some python code from Xiaomin Chen that solves one test case:

```
def Gcd(a, b):
    if b == 0:
        return a
    return Gcd(b, a % b)

def Solve(L):
    y = L[0]
    L1 = [abs(x - y) for x in L]
    g = reduce(Gcd, L1)
    if y % g == 0:
        return 0
    else:
        return g - (y % g)
```

Useful concepts: [Greatest common divisor](#), [Euclidean algorithm](#), [Arbitrary precision arithmetic](#).

Analysis: Snapper Chain

The difficult part was understanding how a single Snapper works. Each Snapper can be in one of two states -- On or Off. Also, each snapper can either be powered or unpowered. The first Snapper is always powered because it is plugged into the power socket in the wall. The i 'th Snapper is powered if and only if the $(i-1)$ 'th Snapper is powered and On. Snapping your fingers changes the state of each powered snapper (from On to Off, or from Off to On).

With these rules in mind, let's represent the On/Off state of the Snappers by a sequence of bits, 1 meaning On. If we list the bits right-to-left, we get a binary integer. Initially, the integer has value 0. Similarly, we can write down the binary integer for the powered/unpowered state of each Snapper. Initially, this integer is 1 because only the rightmost Snapper is powered.

Snapping your fingers is equivalent to doing an XOR of the On/Off integer and the powered/unpowered integer and putting the result into the On/Off integer. After that, we update the powered/unpowered bits according to the rule above,

For example, let's say that the On/Off number is now 10100011111. This means that the powered/unpowered number is 00000111111. When we XOR these two numbers, we get the new value of the On/Off number: 10100100000.

The interesting thing is that these two updates are equivalent to a simple increment of the On/Off integer! Snapping your fingers adds 1 to the On/Off integer, and we do not even need to care about the powered/unpowered integer.

The solution to the problem is then very simple. The answer is "ON" if and only if the rightmost N bits of K are 1.

Analysis: Theme Park

At first glance, this problem appears to be a straightforward simulation: keep adding groups until you run out space on the roller coaster (or run out of groups), then re-form the queue and start again. Repeat this **R** times and you're done.

For the Small, that was enough; and we got more than a few questions during the contest from contestants thinking that they were unable to solve the Large because their computers were so slow. The fact is that, with limits so large -- up to 10^3 groups queuing for 10^9 rides -- you need to come up with a smarter algorithm to solve the problem, since that one is $O(NR)$.

Optimization One

When you're sending the roller coaster out for 10^9 rides, you've got to expect that a few of them are going to be the same. If you store the queue of groups as an unchanging array, with a pointer to the front, then every time you send out a ride *s*, you could make a note of where it ended, given where it started. Then the next time you see a ride starting with the same group in the queue, you can do a quick lookup rather than iterating through all the groups.

That speeds up the algorithm by a factor of 10^3 in the worst case, leaving us with $O(R)$ operations. There are some other ways of speeding up the calculation for any given roller coaster run: for example, you could make an array that makes it $O(1)$ to calculate how many people are in the range [group_a, group_b] and then binary search to figure out how many groups get to go in $O(\log(N))$ time. That gives a total of $O(R \log N)$ operations.

Optimization Two

As we observed in Optimization One, you're going to see a lot of repetition between rides. You're also going to see a lot of repetition between groups of rides. In the example in the problem statement, the queue was made up of groups of size [1, 4, 2, 1]. 6 people get to go at once. Let's look at how the queue changes between rides:

```
1, 4, 2, 1 [5]
2, 1, 1, 4 [4]
4, 2, 1, 1 [6]
1, 1, 4, 2 [6]
2, 1, 1, 4 [4]
4, 2, 1, 1 [6]
1, 1, 4, 2 [6]
```

As you may have noticed, there's a *cycle* of length three: starting from the second run, every third queue looks the same. We make 16 Euros when that happens, which means we'll be making 16 Euros every 3 runs until the roller coaster stops rolling.

So if the roller coaster is set to go 10^9 times: the first time it makes 5 Euros; then there are 999999999 runs left; and every three of those makes 16 Euros. 3 divides 999999999 evenly -- if it didn't, we'd have to do some extra work at the end -- so we make $5 + (999999999 / 3 * 16) = 5333333333$ Euros in total.

It turns out that a cycle *must* show up within the first $N+1$ rides, because there are only **N** different states the queue can be in (after **N**, you have to start repeating). So you only have to

simulate **N** rides, each of which takes $O(N)$ time in the worst case, before finding the cycle: that's an $O(N^2)$ solution.

Optimization Three

Either of the optimizations above should be enough. But if you're a real speed demon, you can squeeze out a little more efficiency by combining the binary search that we mentioned briefly in Optimization One with the cycle detection from Optimization Two, bringing our running time down to $O(N \log N)$. An alternate optimization can bring us down to $O(N)$; we'll leave that as an exercise for the reader. Visit our [Google Group](#) to discuss it with the other contestants!